LAPORAN PRAKTIKUM

POSTTEST 6

ALGORITMA PEMROGRAMAN LANJUT

Heiza Rizki Pratama

2509106019

A'25

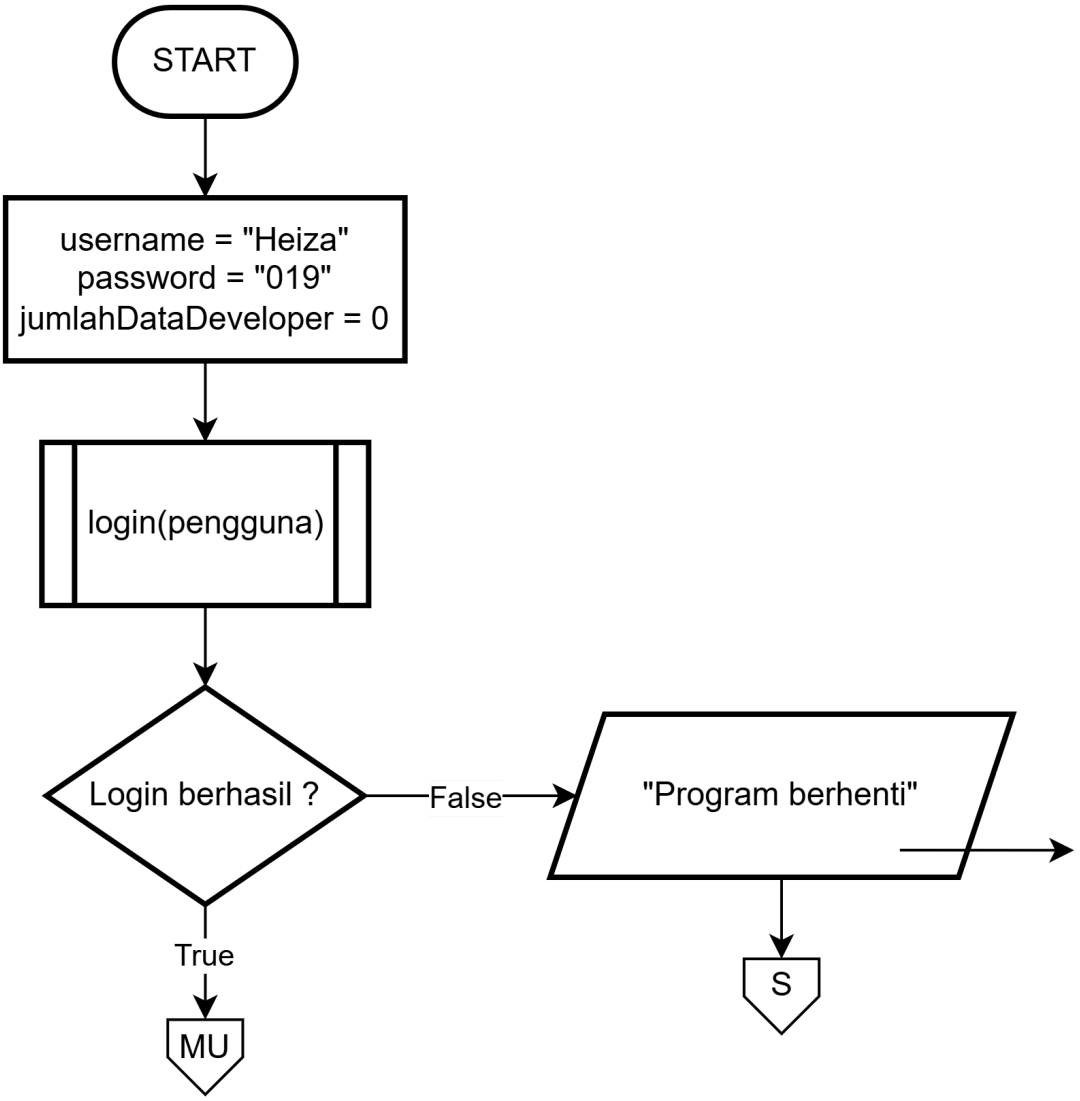
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

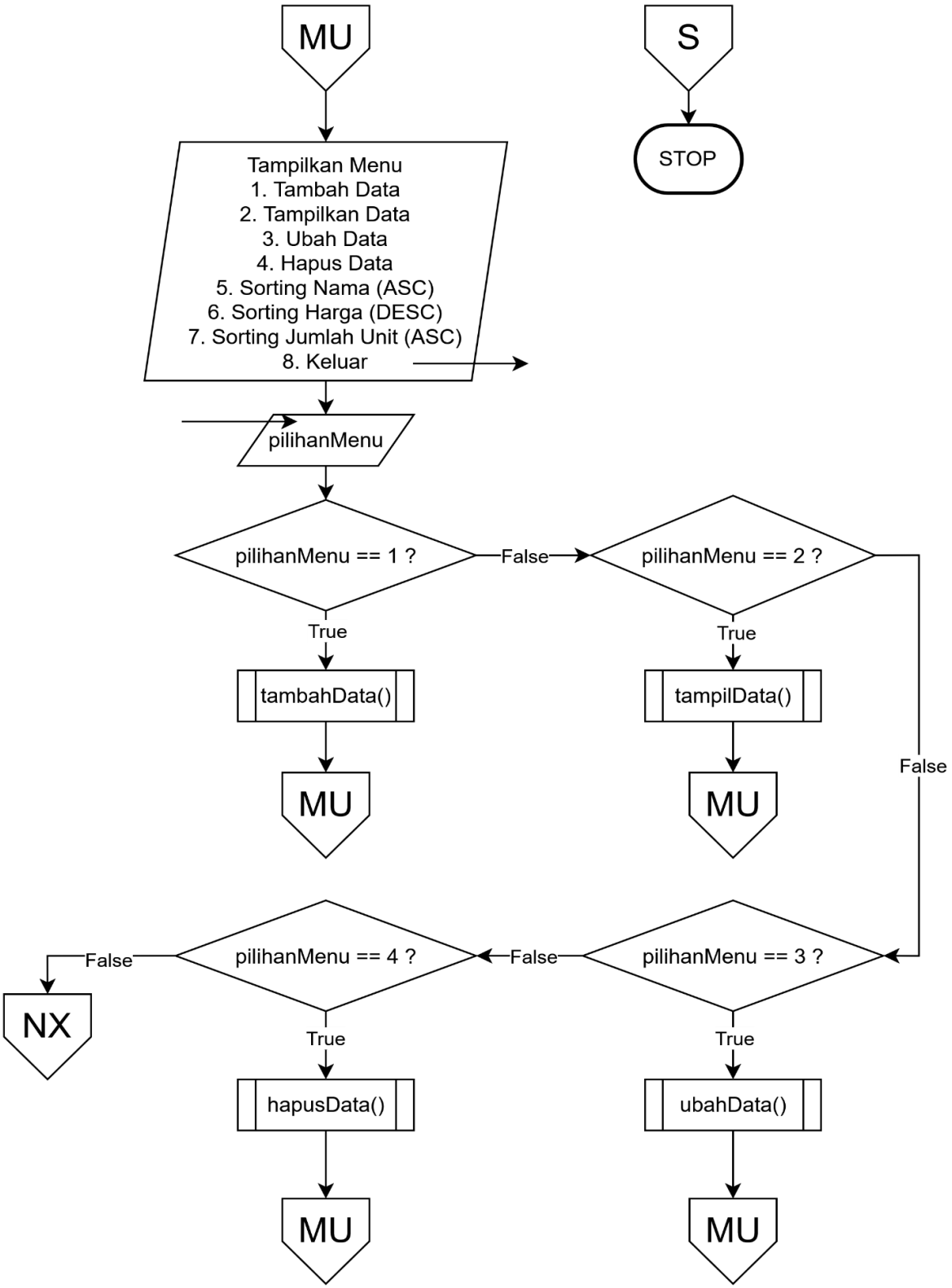
1. Flowchart



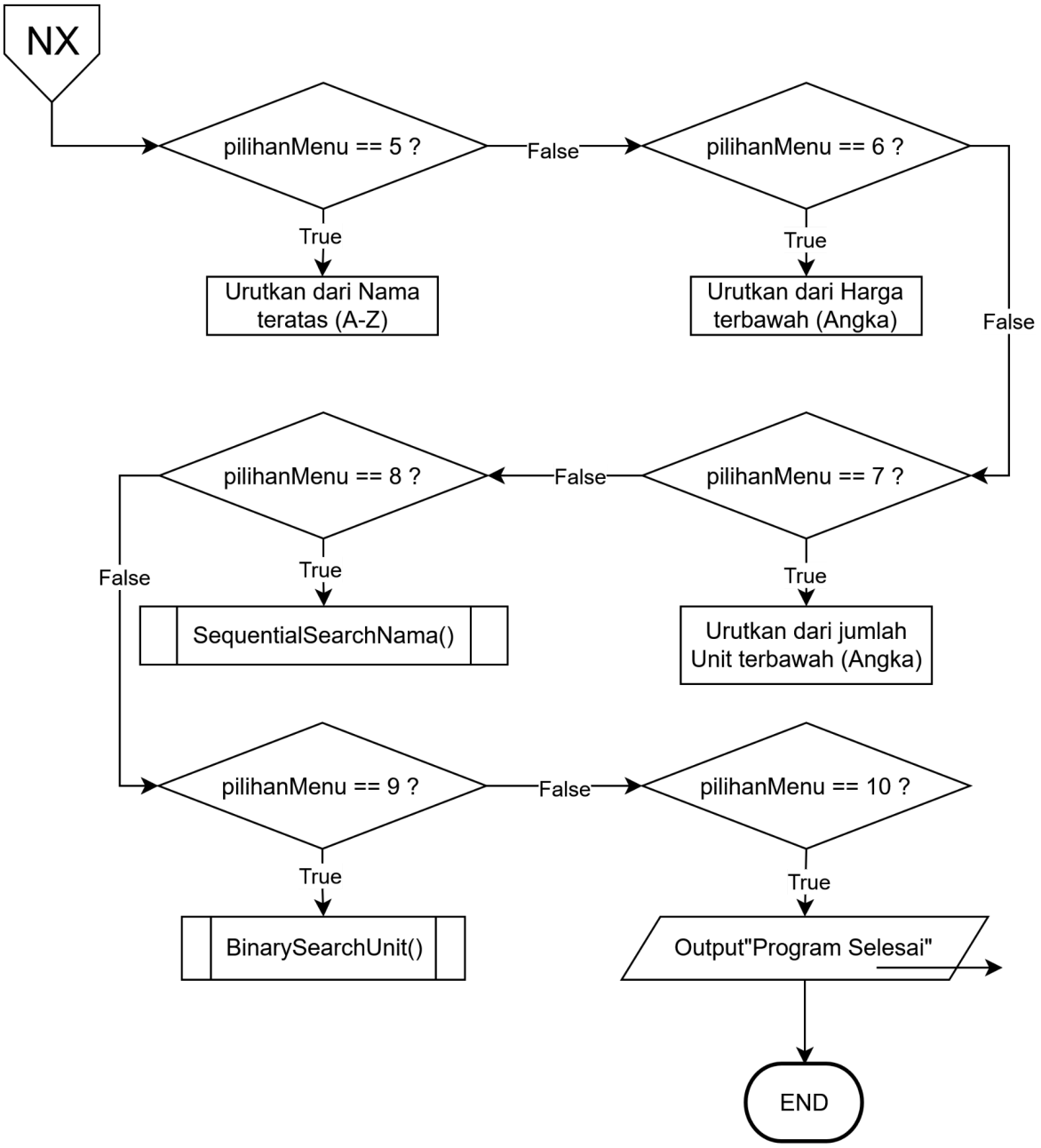
Gambar 1.1

Pada Flowchart diatas ini diperlihatkan proses awal program dan proses login pengguna sebelum dapat mengakses sistem.
Program dimulai dari simbol START. Setelah itu program melakukan inisialisasi beberapa variabel, yaitu menetapkan nilai awal username = "Heiza", password = "019", dan jumlahDataDeveloper = 0. Variabel ini digunakan sebagai data awal untuk proses autentikasi dan penyimpanan jumlah data developer.
Selanjutnya program memanggil subproses login(pengguna) yang berfungsi untuk menerima input username dan password dari pengguna. Proses ini menggunakan simbol predefined process, yang menandakan bahwa proses login didefinisikan dalam prosedur atau fungsi terpisah.

Setelah proses login dilakukan, program melakukan percabangan keputusan dengan kondisi “Login berhasil?”. Jika kondisi bernilai False (login gagal), maka program menampilkan pesan “Program berhenti” dan alur program menuju simbol STOP, yang berarti program tidak dapat dilanjutkan.
Namun jika kondisi bernilai True (login berhasil), maka alur program akan diarahkan ke connector MU, yang menghubungkan flowchart ini dengan flowchart berikutnya yaitu bagian menu utama program.
Dengan demikian, flowchart ini berfungsi sebagai mekanisme keamanan awal program untuk memastikan hanya pengguna yang memiliki username dan password yang benar yang dapat mengakses sistem.


MU
S
STOP
Tampilkan Menu
1. Tambah Data
2. Tampilkan Data
3. Ubah Data
4. Hapus Data
5. Sorting Nama (ASC)
6. Sorting Harga (DESC)
7. Sorting Jumlah Unit (ASC)
8. Keluar
pilihanMenu
pilihanMenu == 1 ?
False
pilihanMenu == 2 ?
True
True
tambahData()
tampilData()
MU
MU
False
pilihanMenu == 4 ?
False
pilihanMenu == 3 ?
False
NX
True
True
hapusData()
ubahData()
MU
MU

Gambar 1.2



Gambar 1.3

Menu Utama (Main Menu) merupakan bagian inti dari program yang berfungsi sebagai pusat kontrol seluruh fitur dalam sistem manajemen data developer perumahan. Pada bagian ini, pengguna diberikan berbagai pilihan menu yang dapat digunakan untuk melakukan operasi terhadap data, seperti menambah, menampilkan, mengubah, menghapus, melakukan pengurutan (sorting), serta
pencarian (searching).

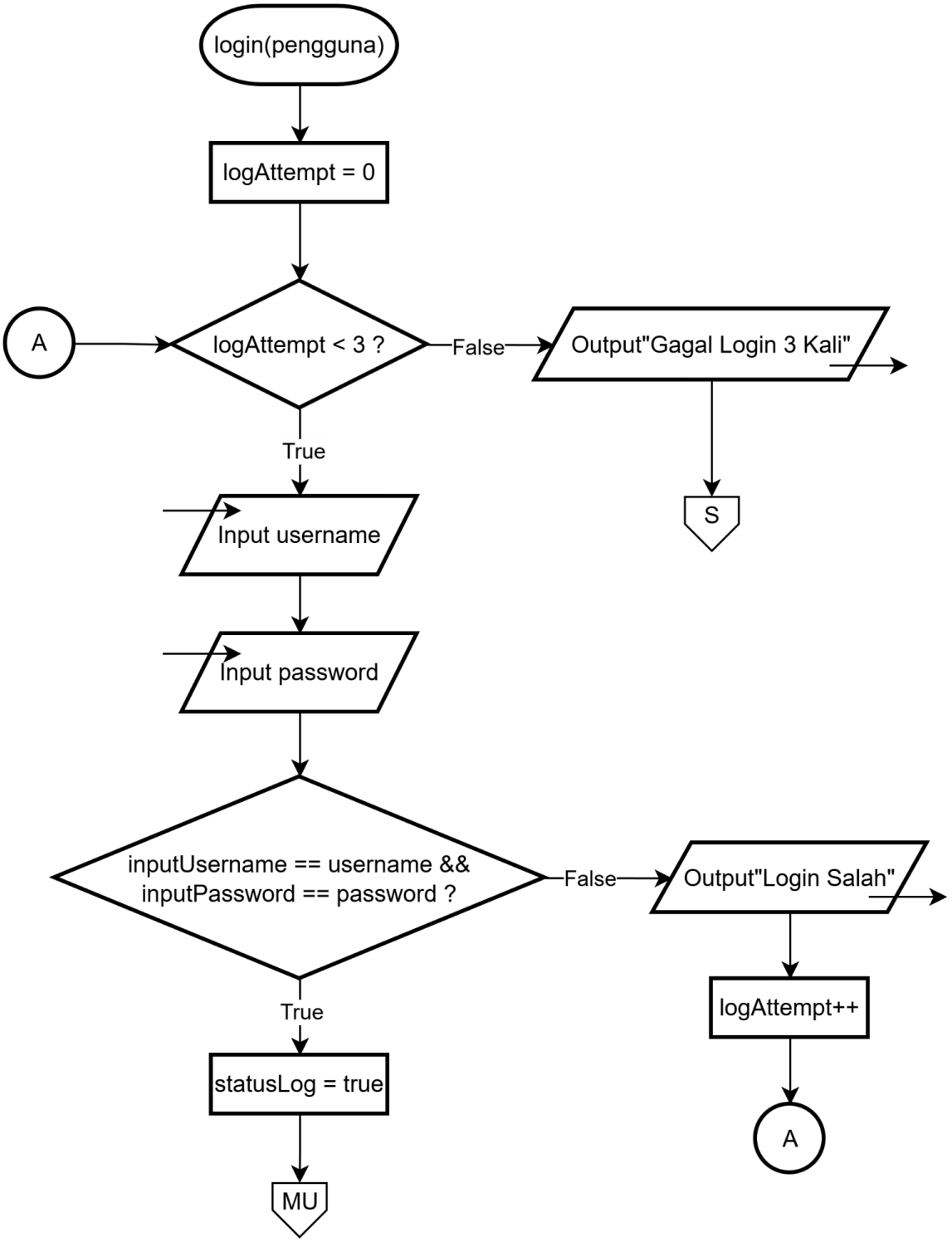
Alur program pada menu utama dimulai setelah pengguna berhasil melakukan proses login. Program kemudian menampilkan daftar menu yang berisi beberapa pilihan, yaitu:

1. Tambah Data
2. Tampilkan Data
3. Ubah Data
4. Hapus Data
5. Sorting Nama (Ascending)
6. Sorting Harga (Descending)
7. Sorting Jumlah Unit (Ascending)
8. Pencarian Nama Developer (Sequential Search)
9. Pencarian Jumlah Unit (Binary Search)
10. Keluar

Setelah menu ditampilkan, pengguna diminta untuk memasukkan pilihan sesuai dengan nomor menu yang tersedia. Input tersebut kemudian diproses menggunakan struktur percabangan (if-else) untuk menentukan aksi yang akan dijalankan.
Setiap pilihan menu memiliki fungsi masing-masing, yaitu:

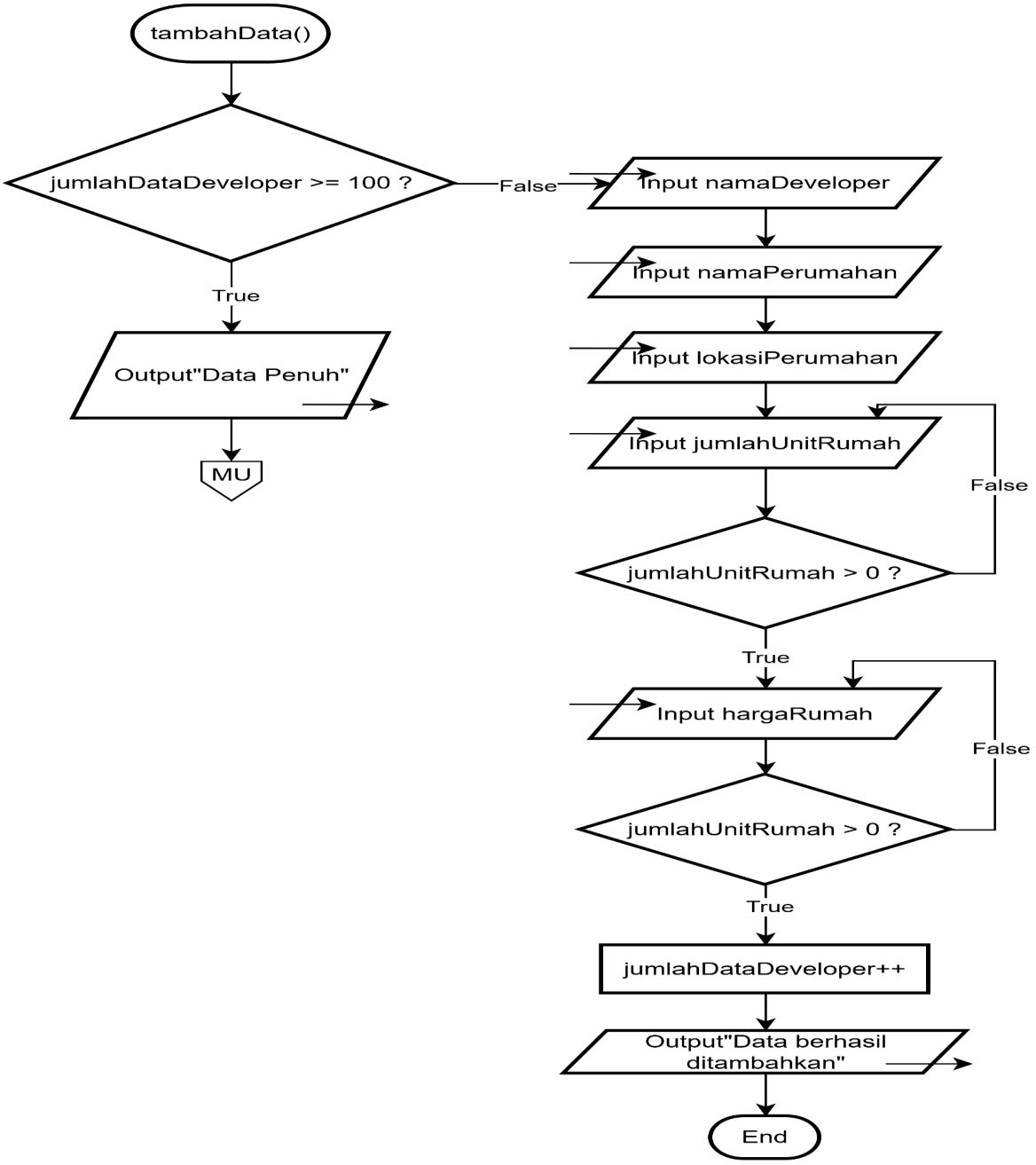
- ➔ Menu 1 (Tambah Data): Digunakan untuk menambahkan data developer baru ke dalam array. Data yang dimasukkan meliputi nama developer, nama perumahan, lokasi, jumlah unit, dan harga rumah.
- ➔ Menu 2 (Tampilkan Data): Menampilkan seluruh data yang telah tersimpan dalam bentuk tabel.
- ➔ Menu 3 (Ubah Data): Memungkinkan pengguna untuk mengubah data tertentu berdasarkan nomor indeks.
- ➔ Menu 4 (Hapus Data): Menghapus data berdasarkan nomor yang dipilih, kemudian menggeser data setelahnya.
- ➔ Menu 5–7 (Sorting): Digunakan untuk mengurutkan data berdasarkan kriteria tertentu, seperti nama, harga, dan jumlah unit.
- ➔ Menu 8–9 (Searching): Digunakan untuk mencari data berdasarkan nama developer atau jumlah unit rumah.
- ➔ Menu 10 (Keluar): Mengakhiri program.

Struktur menu utama menggunakan perulangan do-while, sehingga program akan terus berjalan dan kembali ke menu utama setelah suatu proses selesai dilakukan, selama pengguna belum memilih opsi keluar (menu 10). Hal ini memungkinkan pengguna untuk melakukan berbagai operasi secara berulang tanpa perlu menjalankan ulang program.
Dengan adanya menu utama ini, sistem menjadi lebih terstruktur dan interaktif, serta memudahkan pengguna dalam mengakses seluruh fitur yang tersedia.



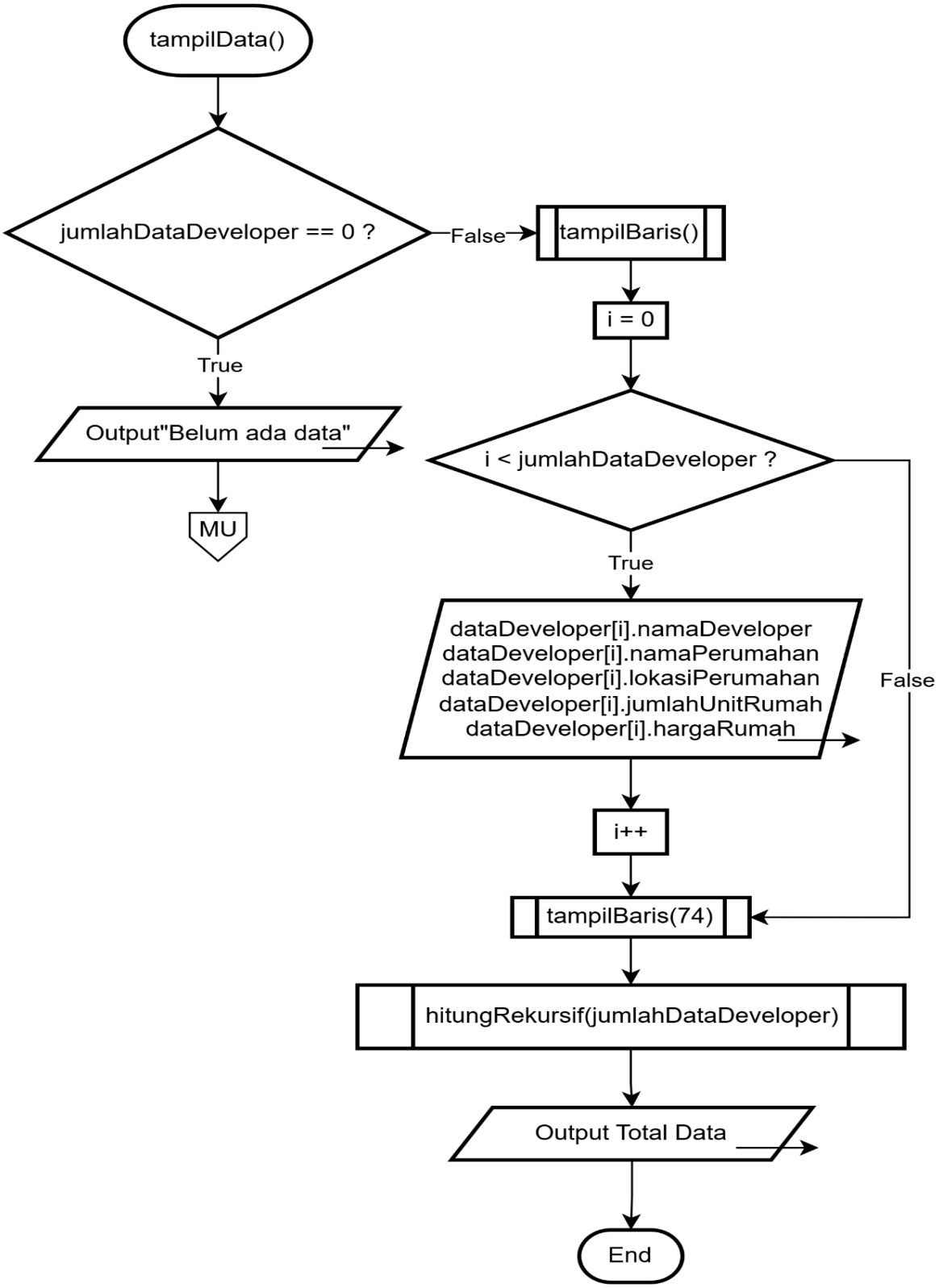
Gambar 1.3

Flowchart login menggambarkan proses autentikasi pengguna dengan batas maksimal 3 kali percobaan. Proses ini menggunakan perulangan dan percabangan untuk menentukan apakah pengguna berhasil masuk atau tidak.



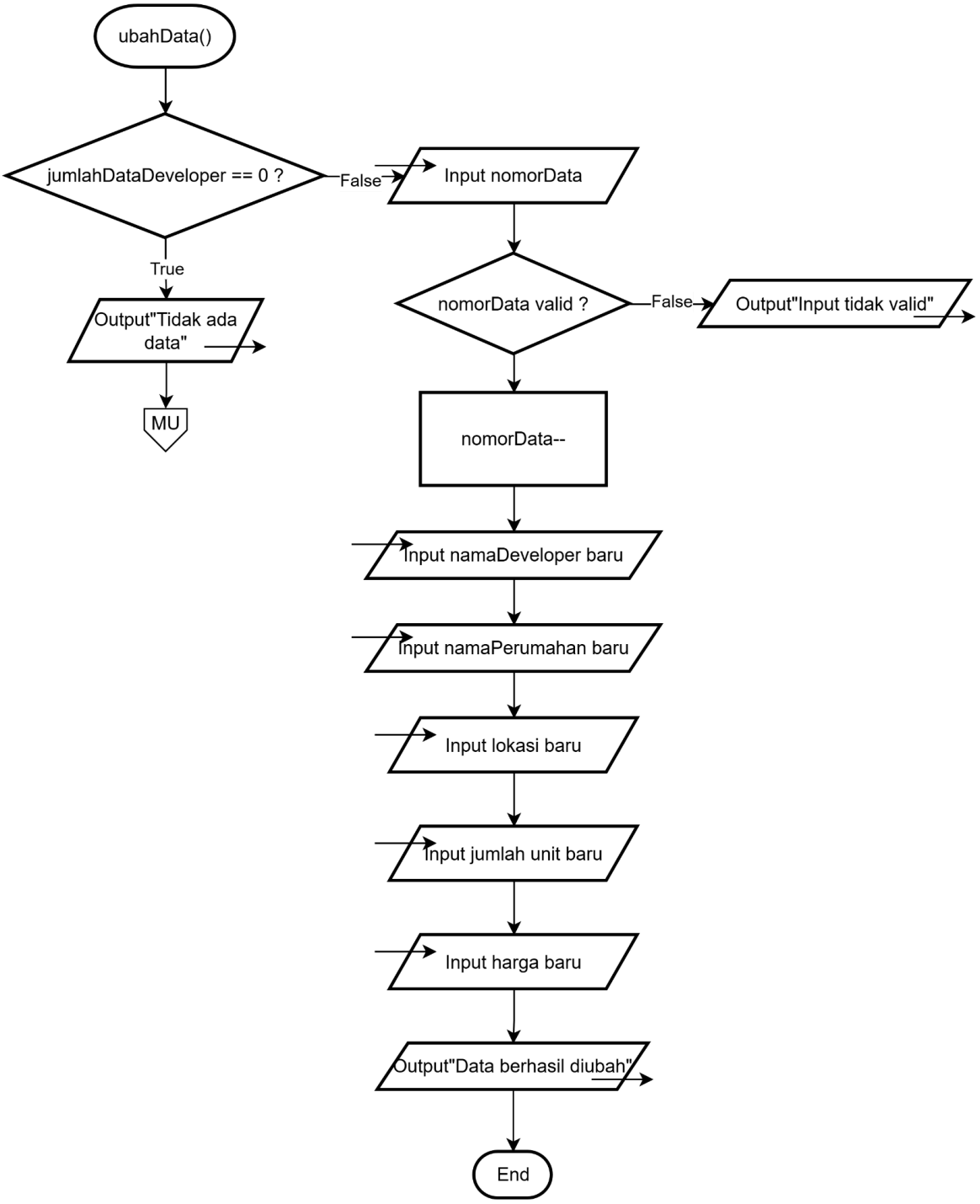
Gambar 1.4

Flowchart tambah data digunakan untuk menambahkan data perumahan ke dalam array dengan batas maksimal 100 data serta dilengkapi validasi input.



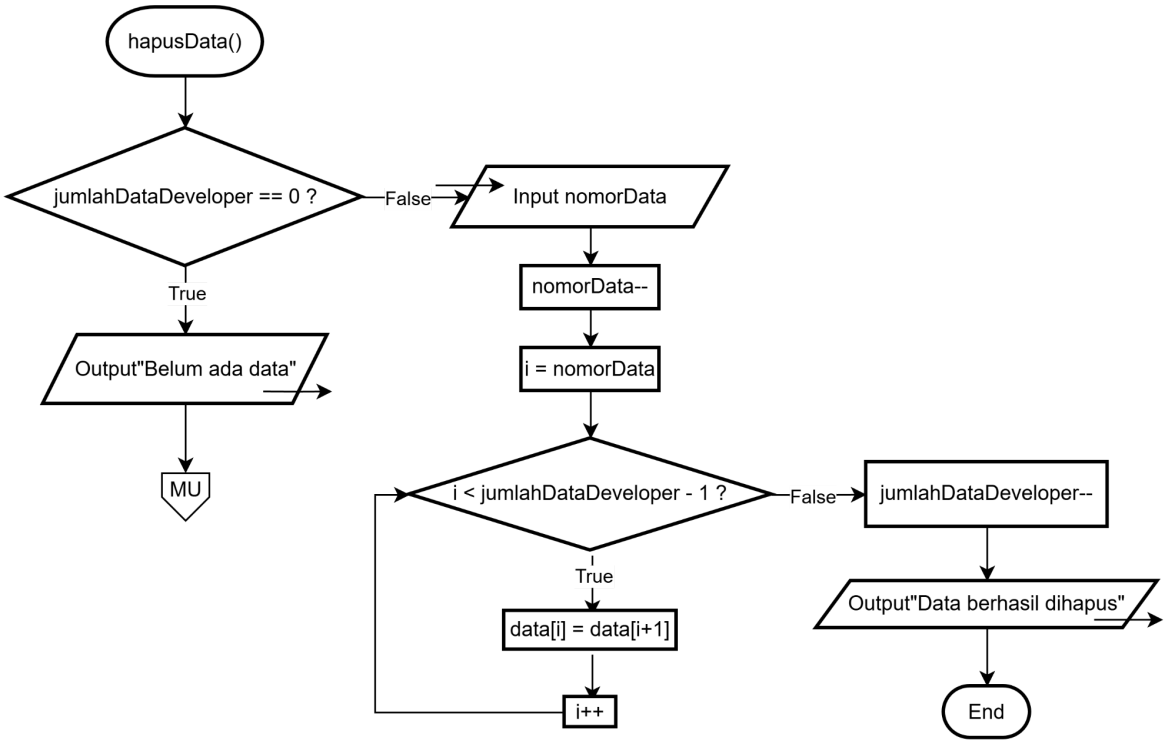
Gambar 1.5

Flowchart tampil data digunakan untuk menampilkan seluruh data developer/perumahan yang tersimpan, serta menampilkan total jumlah data.



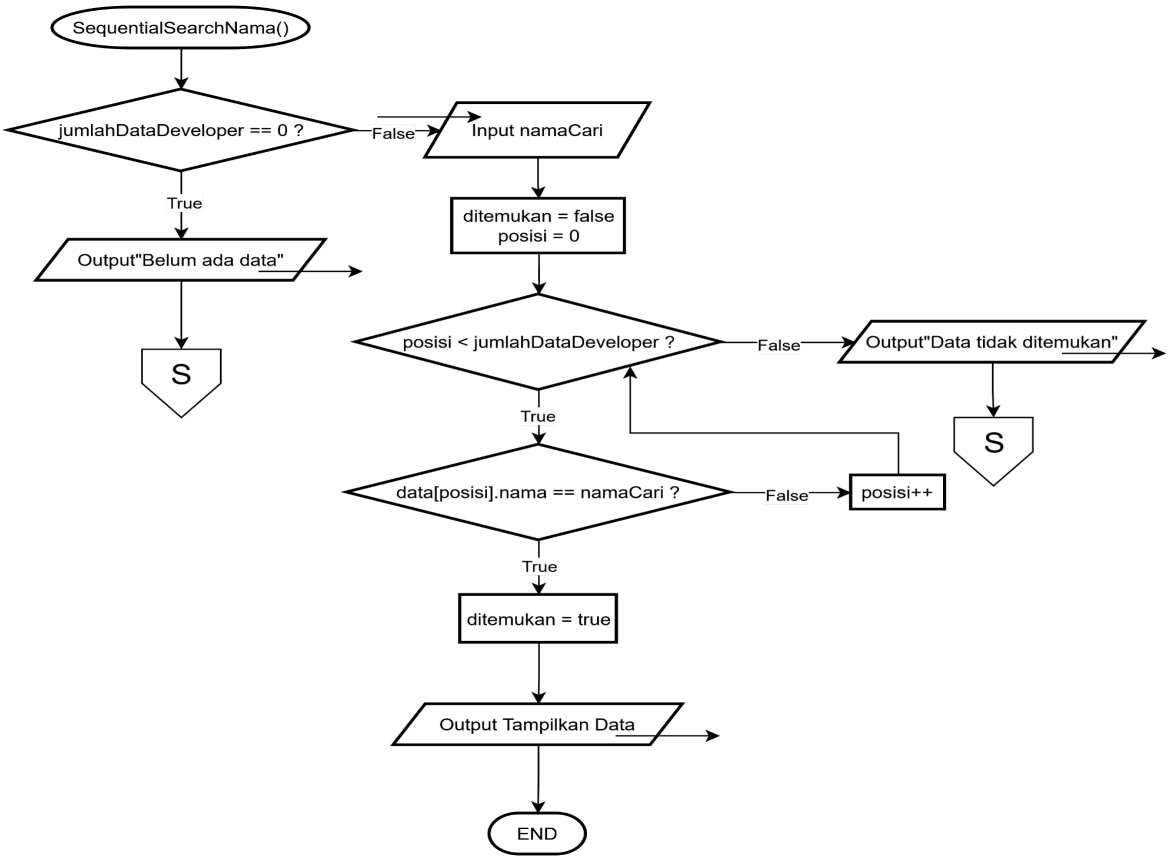
Gambar 1.6

Flowchart ubah data digunakan untuk mengubah data perumahan berdasarkan nomor data yang dipilih oleh user.



Gambar 1.7

Flowchart hapus data digunakan untuk menghapus data perumahan berdasarkan nomor yang dipilih, dengan cara menggeser elemen array dan mengurangi jumlah data.



Gambar 1.8

Sequential Search atau pencarian berurutan merupakan metode pencarian data yang dilakukan dengan cara memeriksa setiap elemen dalam array secara satu per satu, dimulai dari indeks pertama hingga elemen terakhir, sampai data yang dicari ditemukan atau seluruh data telah diperiksa.

Pada program ini, Sequential Search digunakan untuk mencari data berdasarkan nama developer.

Proses pencarian dimulai dengan pengecekan apakah data tersedia atau tidak. Jika jumlah data sama dengan nol, maka program akan langsung menampilkan pesan bahwa data belum tersedia, dan proses pencarian dihentikan.

Jika data tersedia, pengguna diminta untuk memasukkan nama developer yang ingin dicari. Setelah itu, program melakukan inisialisasi variabel, yaitu:

- ➔ ditemukan diset ke false sebagai penanda apakah data ditemukan atau tidak
- ➔ posisi diset ke 0 sebagai indeks awal pencarian

Selanjutnya, program masuk ke proses perulangan (loop) dengan kondisi selama posisi masih lebih kecil dari jumlah data. Pada setiap iterasi, program akan membandingkan nilai nama developer pada indeks saat ini dengan nama yang dicari.

Jika ditemukan kecocokan:

- ➔ Variabel ditemukan diubah menjadi true
- ➔ Data yang sesuai ditampilkan ke layar
- ➔ Proses pencarian dihentikan

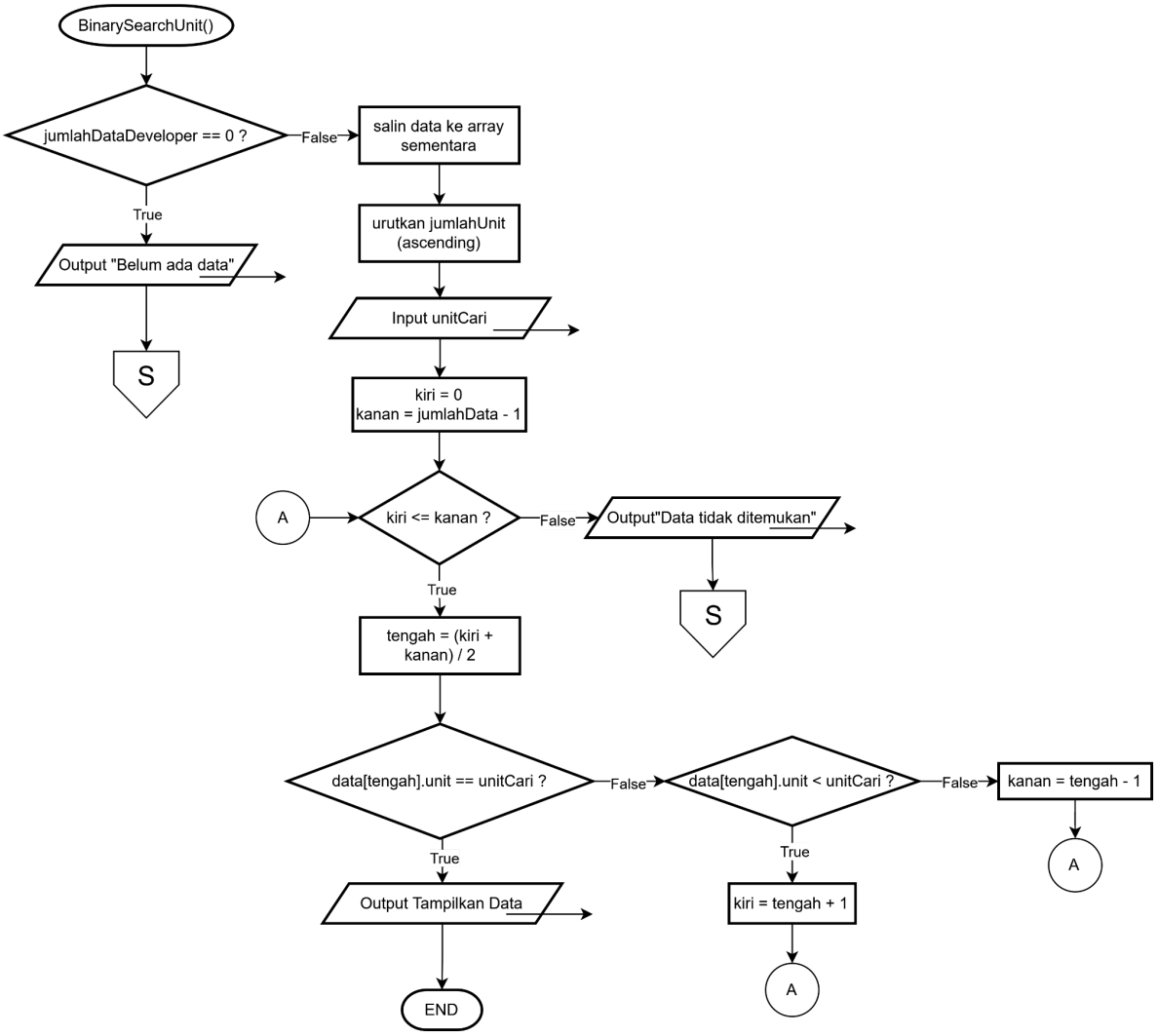
Jika tidak cocok:

- ➔ Nilai posisi akan ditambah satu

➔ Proses kembali ke iterasi berikutnya

Apabila seluruh data telah diperiksa dan tidak ditemukan kecocokan, maka program akan menampilkan pesan bahwa data tidak ditemukan.

Metode Sequential Search memiliki kelebihan yaitu mudah diimplementasikan dan tidak memerlukan data dalam kondisi terurut. Namun, kekurangannya adalah kurang efisien untuk jumlah data yang besar karena harus memeriksa satu per satu.



Gambar 1.9

Binary Search merupakan metode pencarian yang lebih efisien dibandingkan Sequential Search, karena menggunakan teknik pembagian data menjadi dua bagian secara berulang. Namun, metode ini memiliki syarat utama, yaitu data harus dalam keadaan terurut (sorted). Pada program ini, Binary Search digunakan untuk mencari data berdasarkan jumlah unit rumah.

Proses dimulai dengan pengecekan apakah data tersedia. Jika tidak ada data, maka program langsung menampilkan pesan bahwa data belum tersedia.

Jika data tersedia, langkah pertama yang dilakukan adalah menyalin data ke array sementara. Hal ini bertujuan agar proses pengurutan tidak mengubah data asli.

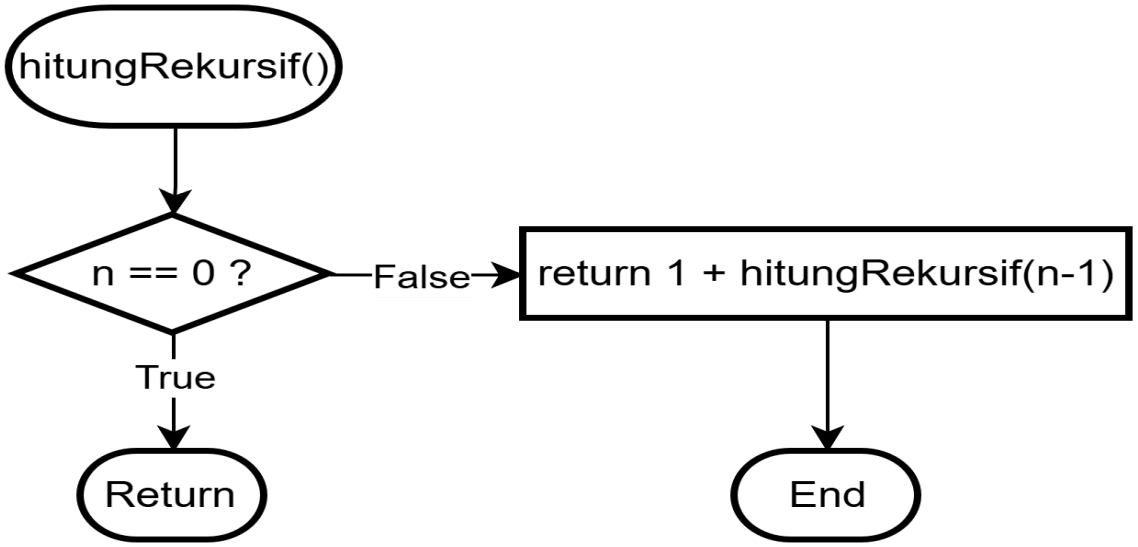
Selanjutnya, data dalam array sementara diurutkan berdasarkan jumlah unit rumah secara ascending menggunakan algoritma insertion sort. Proses pengurutan ini sangat penting karena Binary Search hanya dapat bekerja dengan benar pada data yang sudah terurut.
Setelah itu, pengguna diminta untuk memasukkan jumlah unit yang ingin dicari. Kemudian dilakukan inisialisasi:

- ➔ kiri sebagai indeks awal (0)
- ➔ kanan sebagai indeks akhir (jumlah data - 1)

Proses pencarian dilakukan dengan langkah berikut:

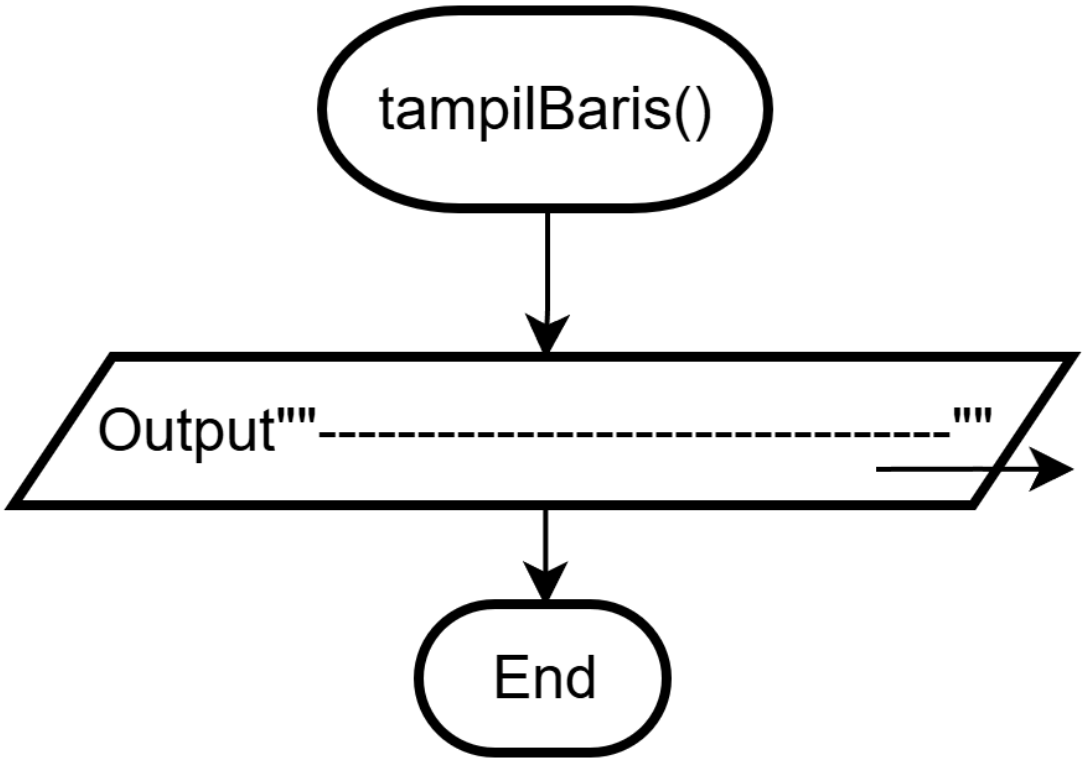
- ➔ Hitung nilai tengah dengan rumus:
  tengah = (kiri + kanan) / 2
- ➔ Bandingkan nilai pada posisi tengah dengan nilai yang dicari:
  - ◆ Jika sama, data ditemukan dan langsung ditampilkan
  - ◆ Jika nilai tengah lebih kecil dari target, maka pencarian dilanjutkan ke bagian kanan dengan menggeser nilai kiri
  - ◆ Jika nilai tengah lebih besar, maka pencarian dilanjutkan ke bagian kiri dengan menggeser nilai kanan
- ➔ Proses ini diulang selama nilai kiri masih kurang dari atau sama dengan nilai kanan.

Jika setelah perulangan selesai data tidak ditemukan, maka program akan menampilkan pesan bahwa data tidak ditemukan.
Binary Search memiliki kelebihan dalam hal efisiensi karena dapat mengurangi jumlah pencarian secara signifikan, dengan kompleksitas waktu O(log n). Namun, metode ini membutuhkan proses tambahan berupa pengurutan data sebelum pencarian dilakukan.



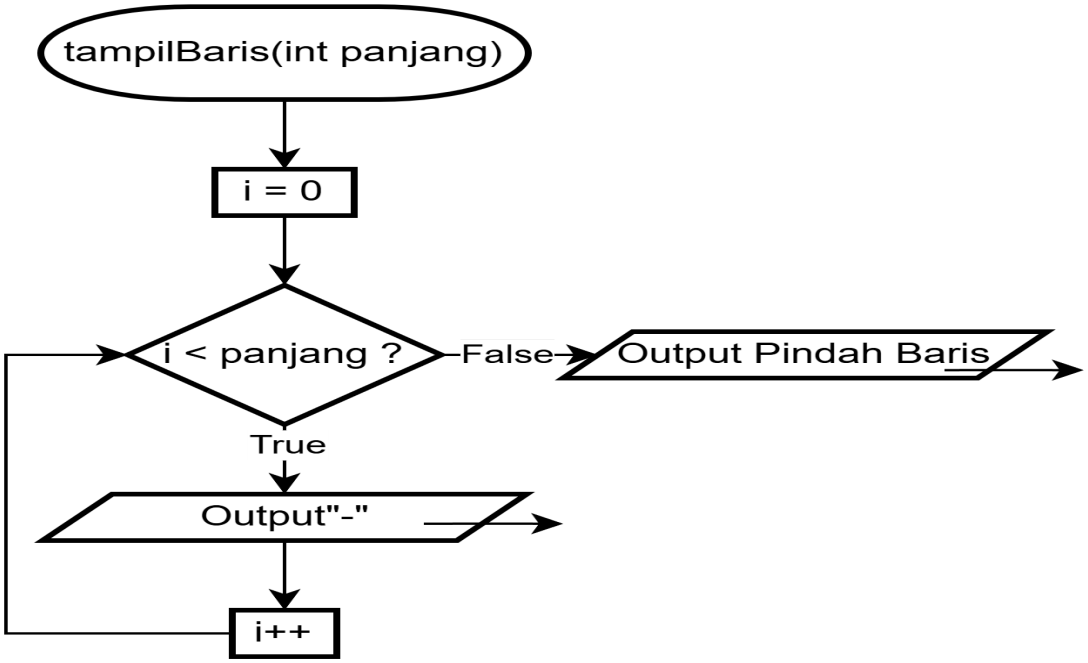
Gambar 2.0

Flowchart hitungRekursif() digunakan untuk menghitung jumlah data secara rekursif, yaitu dengan memanggil fungsi itu sendiri hingga mencapai kondisi dasar.



Gambar 2.1

Menampilkan garis pemisah panjang secara langsung.



Gambar 1.10

Menampilkan garis pemisah sesuai jumlah (panjang) dengan perulangan.

2. Deskripsi Singkat Program

Program ini merupakan Sistem Manajemen Data Developer Perumahan yang dibuat menggunakan bahasa pemrograman C++. Program diawali dengan proses login pengguna yang mengharuskan pengguna memasukkan username dan password yang telah ditentukan sebelumnya. Pengguna diberikan maksimal tiga kali percobaan login. Jika login gagal sebanyak tiga kali, maka program akan berhenti.

Setelah berhasil login, pengguna akan masuk ke menu utama yang menyediakan beberapa fitur untuk mengelola data perumahan yang dikembangkan oleh developer. Program ini menerapkan konsep CRUD (Create, Read, Update, Delete) pada data perumahan yang disimpan dalam bentuk array struktur.

Fitur-fitur yang tersedia dalam program ini meliputi:

1. Tambah Data Perumahan (Create) untuk menambahkan data developer dan perumahan baru.
2. Tampilkan Data Perumahan (Read) untuk menampilkan seluruh data perumahan yang telah disimpan.
3. Ubah Data Perumahan (Update) untuk memperbarui data perumahan yang sudah ada.
4. Hapus Data Perumahan (Delete) untuk menghapus data perumahan tertentu dari daftar.
5. Keluar (Exit) untuk mengakhiri program.

Program ini juga dilengkapi dengan validasi input untuk memastikan data yang dimasukkan pengguna sesuai dengan format yang diharapkan, seperti memastikan jumlah unit rumah dan harga rumah bernilai positif.

3. Source Code

```
struct User {
 string username; string password;
};
struct Developer {
 string namaDeveloper; string namaPerumahan; string
lokasiPerumahan; int jumlahUnitRumah; long hargaRumah;
};
```

```
void tampilBaris() {
    cout <<
"-----------------------------------------------------------------
---------" << endl;
}
```

```
void tampilBaris(int panjang) {
    for (int i = 0; i < panjang; i++) cout << "-";
    cout << endl;
}

int hitungRekursif(int jumlahDataDeveloper) {
    if (jumlahDataDeveloper == 0) return 0;
    return 1 + hitungRekursif(jumlahDataDeveloper - 1);
}
```

```
void bubbleSortNamaAscending(Developer *dataDeveloper, int
jumlahDataDeveloper) {
    for (int i = 0; i < jumlahDataDeveloper - 1; i++) {
        for (int j = 0; j < jumlahDataDeveloper - i - 1; j++) {
            if (dataDeveloper[j].namaDeveloper > dataDeveloper[j +
1].namaDeveloper) {
                Developer temp = dataDeveloper[j];
                dataDeveloper[j] = dataDeveloper[j + 1];
                dataDeveloper[j + 1] = temp;
            }
        }
    }
}

void selectionSortHargaDescending(Developer *dataDeveloper, int
jumlahDataDeveloper) {
    for (int i = 0; i < jumlahDataDeveloper - 1; i++) {
        int maxIndex = i;
        for (int j = i + 1; j < jumlahDataDeveloper; j++) {
            if (dataDeveloper[j].hargaRumah >
dataDeveloper[maxIndex].hargaRumah)
                maxIndex = j;
        }
        Developer temp = dataDeveloper[i];
        dataDeveloper[i] = dataDeveloper[maxIndex];
        dataDeveloper[maxIndex] = temp;
    }
}

void insertionSortUnitAscending(Developer *dataDeveloper, int
```

```
jumlahDataDeveloper) {
    for (int i = 1; i < jumlahDataDeveloper; i++) {
        Developer kunci = dataDeveloper[i];
        int j = i - 1;
        while (j >= 0 && dataDeveloper[j].jumlahUnitRumah >
kunci.jumlahUnitRumah) {
            dataDeveloper[j + 1] = dataDeveloper[j];
            j--;
        }
        dataDeveloper[j + 1] = kunci;
    }
}
```

```
void sequentialSearchNama(Developer *dataDeveloper, int
jumlahDataDeveloper) {
    system("cls");
    cout << "CARI DATA BERDASARKAN NAMA DEVELOPER" << endl;

    if (jumlahDataDeveloper == 0) {
        cout << "Belum ada data." << endl;
        system("pause");
        return;
    }

    string namaCari;
    cout << "Masukkan nama developer yang dicari: ";
    getline(cin, namaCari);

    bool ditemukan = false;
    int indeksDitemukan = -1;

    for (int posisi = 0; posisi < jumlahDataDeveloper; posisi++) {
        if (dataDeveloper[posisi].namaDeveloper == namaCari) {
            ditemukan = true;
            indeksDitemukan = posisi;
            break;
        }
    }

    if (ditemukan) {
```

```
        cout << "Data ditemukan pada nomor " << indeksDitemukan +
1 << endl;
        cout << "Nama Developer  : " <<
dataDeveloper[indeksDitemukan].namaDeveloper << endl;
        cout << "Nama Perumahan  : " <<
dataDeveloper[indeksDitemukan].namaPerumahan << endl;
        cout << "Lokasi          : " <<
dataDeveloper[indeksDitemukan].lokasiPerumahan << endl;
        cout << "Jumlah Unit     : " <<
dataDeveloper[indeksDitemukan].jumlahUnitRumah << endl;
        cout << "Harga Rumah     : " <<
dataDeveloper[indeksDitemukan].hargaRumah << endl;
    } else {
        cout << "Data dengan nama \"" << namaCari << "\" tidak
ditemukan." << endl;
    }

    system("pause");
}

void binarySearchUnit(Developer *dataDeveloper, int
jumlahDataDeveloper) {
    system("cls");
    cout << "CARI DATA BERDASARKAN JUMLAH UNIT RUMAH" << endl;

    if (jumlahDataDeveloper == 0) {
        cout << "Belum ada data." << endl;
        system("pause");
        return;
    }

    Developer dataSementara[100];
    for (int i = 0; i < jumlahDataDeveloper; i++) dataSementara[i]
= dataDeveloper[i];
    insertionSortUnitAscending(dataSementara,
jumlahDataDeveloper);

    int unitCari;
    cout << "Masukkan jumlah unit yang dicari: ";
    cin >> unitCari;
    cin.ignore(1000, '\n');
```

```
    int posisiKiri = 0, posisiKanan = jumlahDataDeveloper - 1,
posisiTengah;
    bool ditemukan = false;
    int indeksDitemukan = -1;

    while (posisiKiri <= posisiKanan) {
        posisiTengah = (posisiKiri + posisiKanan) / 2;
        if (dataSementara[posisiTengah].jumlahUnitRumah ==
unitCari) {
            ditemukan = true;
            indeksDitemukan = posisiTengah;
            break;
        } else if (dataSementara[posisiTengah].jumlahUnitRumah <
unitCari) {
            posisiKiri = posisiTengah + 1;
        } else {
            posisiKanan = posisiTengah - 1;
        }
    }

    if (ditemukan) {
        cout << "Data ditemukan!" << endl;
        cout << "Nama Developer  : " <<
dataSementara[indeksDitemukan].namaDeveloper << endl;
        cout << "Nama Perumahan  : " <<
dataSementara[indeksDitemukan].namaPerumahan << endl;
        cout << "Lokasi          : " <<
dataSementara[indeksDitemukan].lokasiPerumahan << endl;
        cout << "Jumlah Unit     : " <<
dataSementara[indeksDitemukan].jumlahUnitRumah << endl;
        cout << "Harga Rumah     : " <<
dataSementara[indeksDitemukan].hargaRumah << endl;
    } else {
        cout << "Data dengan jumlah unit " << unitCari << " tidak
ditemukan." << endl;
    }

    system("pause");
}
```

```
bool login(User *pengguna) {
    string inputUsername, inputPassword;
    int percobaan = 0;
    bool statusLogin = false;

    while (percobaan < 3) {
        system("cls");
        cout << "===== LOGIN SISTEM =====" << endl;
        cout << "Masukkan Username (Nama): ";
        cin >> inputUsername;

        cout << "Masukkan Password (NIM): ";
        inputPassword = "";
        char karakter;
        while ((karakter = _getch()) != 13) {
            if (karakter == 8) {
                if (!inputPassword.empty()) { cout << "\b \b";
inputPassword.pop_back(); }
            } else {
                cout << "*";
                inputPassword += karakter;
            }
        }
        cout << endl;

        if (inputUsername == pengguna->username && inputPassword
== pengguna->password) {
            statusLogin = true;
            break;
        } else {
            cout << "Login Salah" << endl;
            percobaan++;
            system("pause");
        }
    }

    if (!statusLogin) cout << "Gagal login 3 kali" << endl;
    return statusLogin;
}
```

```
void tambahData(Developer *dataDeveloper, int
*jumlahDataDeveloper) {
    system("cls");
    cout << "TAMBAH DATA PERUMAHAN YANG DIKEMBANGKAN" << endl;

    if (*jumlahDataDeveloper >= 100) {
        cout << "Data penuh" << endl;
        system("pause");
    } else {
        cout << "Nama Developer   : "; getline(cin,
dataDeveloper[*jumlahDataDeveloper].namaDeveloper);
        cout << "Nama Perumahan   : "; getline(cin,
dataDeveloper[*jumlahDataDeveloper].namaPerumahan);
        cout << "Lokasi Perumahan : "; getline(cin,
dataDeveloper[*jumlahDataDeveloper].lokasiPerumahan);
        cout << "Jumlah Unit Rumah: "; cin >>
dataDeveloper[*jumlahDataDeveloper].jumlahUnitRumah;
        cout << "Harga Rumah      : "; cin >>
dataDeveloper[*jumlahDataDeveloper].hargaRumah;
        cin.ignore(1000, '\n');
        (*jumlahDataDeveloper)++;
        cout << "Data berhasil ditambahkan" << endl;
        system("pause");
    }
}

void tampilData(Developer *dataDeveloper, int jumlahDataDeveloper)
{
    system("cls");
    cout << "DATA PERUMAHAN" << endl;

    if (jumlahDataDeveloper == 0) {
        cout << "Belum ada data." << endl;
    } else {
        tampilBaris();
        cout << "No | Nama Developer | Nama Perumahan | Lokasi |
Jumlah Unit | Harga Rumah" << endl;
        tampilBaris();
        for (int i = 0; i < jumlahDataDeveloper; i++) {
            cout << i + 1 << " | "
                 << dataDeveloper[i].namaDeveloper << " | "
                 << dataDeveloper[i].namaPerumahan << " | "
```

```
                << dataDeveloper[i].lokasiPerumahan << " | "
                << dataDeveloper[i].jumlahUnitRumah << " | "
                << dataDeveloper[i].hargaRumah << endl;
        }
        tampilBaris(74);
        cout << "Total Data : " <<
hitungRekursif(jumlahDataDeveloper) << endl;
    }

    system("pause");
}

void ubahData(Developer *dataDeveloper, int jumlahDataDeveloper) {
    system("cls");
    cout << "UBAH DATA" << endl;
    int nomorData;

    if (jumlahDataDeveloper == 0) {
        cout << "Tidak ada data." << endl;
        system("pause");
    } else {
        cout << "Masukkan nomor data (1-" << jumlahDataDeveloper
<< "): ";
        cin >> nomorData;

        if (nomorData < 1 || nomorData > jumlahDataDeveloper) {
            cout << "Nomor data tidak valid." << endl;
            system("pause");
            return;
        }

        nomorData--;
        cin.ignore(1000, '\n');

        cout << "Nama Developer Baru : "; getline(cin,
dataDeveloper[nomorData].namaDeveloper);
        cout << "Nama Perumahan Baru : "; getline(cin,
dataDeveloper[nomorData].namaPerumahan);
        cout << "Lokasi Baru         : "; getline(cin,
dataDeveloper[nomorData].lokasiPerumahan);
        cout << "Jumlah Unit Baru    : "; cin >>
dataDeveloper[nomorData].jumlahUnitRumah;
```

```
        cout << "Harga Baru          : "; cin >>
dataDeveloper[nomorData].hargaRumah;
        cin.ignore(1000, '\n');

        cout << "Data berhasil diubah" << endl;
        system("pause");
    }
}

void hapusData(Developer *dataDeveloper, int *jumlahDataDeveloper)
{
    system("cls");
    cout << "HAPUS DATA" << endl;
    int nomorData;

    if (*jumlahDataDeveloper == 0) {
        cout << "Belum ada data." << endl;
        system("pause");
    } else {
        cout << "Masukkan nomor data (1-" << *jumlahDataDeveloper
<< "): ";
        cin >> nomorData;

        if (nomorData < 1 || nomorData > *jumlahDataDeveloper) {
            cout << "Nomor data tidak valid." << endl;
            system("pause");
            return;
        }

        nomorData--;
        for (int i = nomorData; i < (*jumlahDataDeveloper) - 1;
i++)
            dataDeveloper[i] = dataDeveloper[i + 1];
        (*jumlahDataDeveloper)--;

        cout << "Data berhasil dihapus" << endl;
        system("pause");
    }
}
```

```
int main() {
    User pengguna;
    pengguna.username = "Heiza";
    pengguna.password = "019";

    if (login(&pengguna) == false) return 0;

    Developer dataDeveloper[100];
    int jumlahDataDeveloper = 0;
    int pilihanMenu;

    do {
        system("cls");
        cout << "SISTEM MANAJEMEN" << endl;
        cout << "1.  Tambah Data" << endl;
        cout << "2.  Tampilkan Data" << endl;
        cout << "3.  Ubah Data" << endl;
        cout << "4.  Hapus Data" << endl;
        cout << "5.  Sorting Nama (Ascending)" << endl;
        cout << "6.  Sorting Harga (Descending)" << endl;
        cout << "7.  Sorting Jumlah Unit (Ascending)" << endl;
        cout << "8.  Cari Nama Developer (Sequential Search)" <<
endl;
        cout << "9.  Cari Jumlah Unit (Binary Search)" << endl;
        cout << "10. Keluar" << endl;
        cout << "Pilih Menu : ";
        cin >> pilihanMenu;
        cin.ignore(1000, '\n');

        if      (pilihanMenu == 1) tambahData(dataDeveloper,
&jumlahDataDeveloper);
        else if (pilihanMenu == 2) tampilData(dataDeveloper,
jumlahDataDeveloper);
        else if (pilihanMenu == 3) ubahData(dataDeveloper,
jumlahDataDeveloper);
        else if (pilihanMenu == 4) hapusData(dataDeveloper,
&jumlahDataDeveloper);
        else if (pilihanMenu == 5) {
bubbleSortNamaAscending(dataDeveloper, jumlahDataDeveloper);
cout << "Sorting nama berhasil" << endl;  system("pause"); }
        else if (pilihanMenu == 6) {
selectionSortHargaDescending(dataDeveloper, jumlahDataDeveloper);
```

```
cout << "Sorting harga berhasil" << endl; system("pause"); }
        else if (pilihanMenu == 7) {
insertionSortUnitAscending(dataDeveloper, jumlahDataDeveloper);
cout << "Sorting unit berhasil" << endl;  system("pause"); }
        else if (pilihanMenu == 8)
sequentialSearchNama(dataDeveloper, jumlahDataDeveloper);
        else if (pilihanMenu == 9) binarySearchUnit(dataDeveloper,
jumlahDataDeveloper);

    } while (pilihanMenu != 10);

    cout << "Program selesai." << endl;
}
```

4. PENJELASAN PROGRAM

Program ini merupakan aplikasi berbasis bahasa C++ yang digunakan untuk mengelola data developer perumahan. Sistem ini dirancang untuk mempermudah pengguna dalam melakukan pengolahan data seperti menambah, menampilkan, mengubah, menghapus, mengurutkan, serta mencari data developer.
Data developer disimpan dalam sebuah array dengan kapasitas maksimal 100 data. Setiap data memiliki beberapa atribut, yaitu:

- Nama Developer
- Nama Perumahan
- Lokasi Perumahan
- Jumlah Unit Rumah
- Harga Rumah

Selain itu, program juga dilengkapi dengan fitur keamanan berupa login, sehingga hanya pengguna yang memiliki username dan password yang dapat mengakses sistem.

4.2 Struktur Data

Program menggunakan struktur (struct) untuk merepresentasikan data developer sebagai berikut:

```
struct Developer {
  string namaDeveloper;
  string namaPerumahan;
  string lokasiPerumahan;
  int jumlahUnitRumah;
  long hargaRumah;
};
```

Struktur ini digunakan untuk menyimpan seluruh informasi terkait developer dalam satu kesatuan data.

4.3 Penjelasan Fungsi-Fungsi Program

4.3.1 Fungsi tampilBaris()

Fungsi ini digunakan untuk menampilkan garis pembatas pada tampilan program agar output lebih rapi dan mudah dibaca.
Terdapat dua versi fungsi:

1. Tanpa parameter → menampilkan garis dengan panjang tetap
2. Dengan parameter → menampilkan garis sesuai panjang yang diinginkan

Fungsi ini tidak mengembalikan nilai dan hanya berfungsi untuk memperindah tampilan.

4.3.2 Fungsi hitungRekursif()

Fungsi ini digunakan untuk menghitung jumlah data menggunakan metode rekursif.
Cara kerja fungsi:

- Jika jumlah data = 0, maka fungsi mengembalikan 0
- Jika tidak, fungsi akan memanggil dirinya sendiri hingga mencapai kondisi dasar

Fungsi ini digunakan untuk menampilkan total data pada menu tampilan data.

4.3.3 Fungsi Sorting

Program menyediakan tiga metode pengurutan data, yaitu:

a. Bubble Sort (Ascending berdasarkan Nama)

Fungsi:

bubbleSortNamaAscending()

Digunakan untuk mengurutkan data berdasarkan nama developer dari A sampai Z.
Metode ini bekerja dengan cara membandingkan dua data yang berdekatan, kemudian menukarnya jika urutannya salah.

---

b. Selection Sort (Descending berdasarkan Harga)

Fungsi:
selectionSortHargaDescending()
Digunakan untuk mengurutkan data berdasarkan harga rumah dari yang tertinggi ke terendah.
Algoritma ini mencari nilai terbesar, kemudian menempatkannya di posisi yang sesuai.

---

c. Insertion Sort (Ascending berdasarkan Jumlah Unit)

Fungsi:
insertionSortUnitAscending()
Digunakan untuk mengurutkan data berdasarkan jumlah unit rumah dari kecil ke besar.
Metode ini juga digunakan sebagai persiapan sebelum melakukan binary search.

4.3.4 Fungsi Sequential Search

Fungsi:
sequentialSearchNama()
Digunakan untuk mencari data developer berdasarkan nama.
Cara kerja:

1. Pengguna memasukkan nama yang ingin dicari
2. Program melakukan pencarian dari data pertama hingga terakhir
3. Jika ditemukan, data akan ditampilkan
4. Jika tidak ditemukan, program akan menampilkan pesan bahwa data tidak ada

Kelebihan:

- Tidak memerlukan data terurut

Kekurangan:

- Kurang efisien untuk jumlah data yang besar

4.3.5 Fungsi Binary Search

Fungsi:
binarySearchUnit()
Digunakan untuk mencari data berdasarkan jumlah unit rumah.
Tahapan proses:

1. Data disalin ke array sementara
2. Data diurutkan terlebih dahulu menggunakan insertion sort
3. Pencarian dilakukan dengan metode pembagian dua (binary search)

Cara kerja:

- Menentukan posisi tengah
- Membandingkan nilai tengah dengan nilai yang dicari
- Mempersempit area pencarian hingga data ditemukan atau tidak ditemukan

Kelebihan:

- Lebih cepat dibandingkan sequential search

Kekurangan:

- Harus menggunakan data yang sudah terurut

4.3.6 Fungsi Login

Fungsi:
login()
Digunakan untuk membatasi akses ke program.
Cara kerja:

- Pengguna memasukkan username dan password
- Password disembunyikan menggunakan karakter *
- Pengguna diberikan maksimal 3 kali percobaan

Jika login berhasil, program akan dilanjutkan ke menu utama. Jika gagal, program akan berhenti.

4.3.7 Fungsi Tambah Data

Fungsi:
tambahData()
Digunakan untuk menambahkan data developer baru ke dalam array.
Program akan:

- Meminta input data dari pengguna
- Menyimpan data ke array
- Menambah jumlah data

4.3.8 Fungsi Tampil Data

Fungsi:
tampilData()
Digunakan untuk menampilkan seluruh data developer dalam bentuk tabel.
Jika tidak ada data, program akan menampilkan pesan bahwa data masih kosong.

4.3.9 Fungsi Ubah Data

Fungsi:
ubahData()
Digunakan untuk mengubah data yang sudah ada.
Pengguna akan:

- Memilih nomor data
- Menginput data baru
- Data lama akan digantikan dengan data baru

4.3.10 Fungsi Hapus Data

Fungsi:
hapusData()
Digunakan untuk menghapus data berdasarkan nomor.
Setelah data dihapus:

- Data setelahnya akan digeser ke depan
- Jumlah data akan berkurang

4.4 Fungsi Main (Menu Utama)

Fungsi main() merupakan pusat kendali program.
Program dimulai dengan:

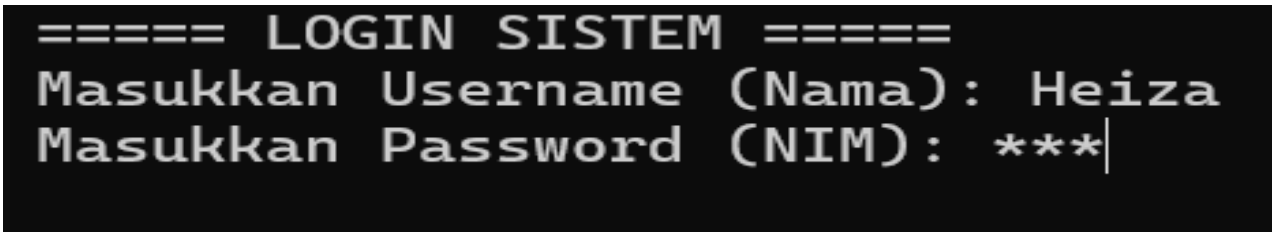
1. Inisialisasi username dan password
2. Proses login
3. Jika login berhasil, program menampilkan menu utama

Menu utama terdiri dari:

1. Tambah Data
2. Tampilkan Data
3. Ubah Data
4. Hapus Data
5. Sorting Nama
6. Sorting Harga
7. Sorting Jumlah Unit
8. Sequential Search
9. Binary Search
10. Keluar

Program menggunakan struktur perulangan do-while sehingga menu akan terus ditampilkan sampai pengguna memilih keluar.

4. OUTPUT PROGRAM



Login

```
SISTEM MANAJEMEN
1.  Tambah Data
2.  Tampilkan Data
3.  Ubah Data
4.  Hapus Data
5.  Sorting Nama (Ascending)
6.  Sorting Harga (Descending)
7.  Sorting Jumlah Unit (Ascending)
8.  Cari Nama Developer (Sequential Search)
9.  Cari Jumlah Unit (Binary Search)
10. Keluar
Pilih Menu :
```

Menu Utama

```
=== TAMBAH DATA PERUMAHAN YANG DIKEMBANGKAN ===
Nama Developer : Lumpus
Nama Perumahan : L corp
Lokasi Perumahan : Jl. setiabudi
Jumlah Unit Rumah : 189
Harga Rumah : 179000000
Data perumahan berhasil ditambahkan
Press any key to continue . . .
```

Create Data Perumahan

```
=== DATA PERUMAHAN YANG SEDANG DIKEMBANGKAN ===
---------------------------------------------------------------------------
No | Developer | Perumahan | Lokasi | Unit Rumah | Harga Rumah
---------------------------------------------------------------------------
1 | Lumpus | L corp | Jl. setiabudi | 189 | 179000000
---------------------------------------------------------------------------
Total Data : 1
Press any key to continue . . .
```

Read Data Perumahan

```
=== UBAH DATA PERUMAHAN ===
Masukkan nomor data yang ingin diubah (1 - 1) : 1
Nama Developer Baru : Luminous
Nama Perumahan Baru : Gugugaga
Lokasi Perumahan Baru : Jl. Lingai Jati
Jumlah Unit Rumah Baru : 190
Harga Rumah Baru : 400000000
Data berhasil diperbarui
Press any key to continue . . .
```

Update Data Perumahan

```
=== HAPUS DATA PERUMAHAN ===
Masukkan nomor data yang ingin dihapus (1 - 1) : 1
Data perumahan berhasil dihapus
Press any key to continue . . .
```

Delete Data Perumahan

```
SISTEM MANAJEMEN
1.  Tambah Data
2.  Tampilkan Data
3.  Ubah Data
4.  Hapus Data
5.  Sorting Nama (Ascending)
6.  Sorting Harga (Descending)
7.  Sorting Jumlah Unit (Ascending)
8.  Cari Nama Developer (Sequential Search)
9.  Cari Jumlah Unit (Binary Search)
10. Keluar
Pilih Menu : 5
Sorting nama berhasil
Press any key to continue . . .
```

```
DATA PERUMAHAN
---------------------------------------------------------------------------
No | Nama Developer | Nama Perumahan | Lokasi | Jumlah Unit | Harga Rumah
---------------------------------------------------------------------------
1 | efad | fass | gsfa | 2141 | 190000000
2 | faga | qte | ewhd | 1263 | 170000000
3 | hdew | ytr | rtrur | 1352 | 165000000
---------------------------------------------------------------------------
Total Data : 3
Press any key to continue . . .
```

Sorting Nama Perumahan

```
SISTEM MANAJEMEN
1.  Tambah Data
2.  Tampilkan Data
3.  Ubah Data
4.  Hapus Data
5.  Sorting Nama (Ascending)
6.  Sorting Harga (Descending)
7.  Sorting Jumlah Unit (Ascending)
8.  Cari Nama Developer (Sequential Search)
9.  Cari Jumlah Unit (Binary Search)
10. Keluar
Pilih Menu : 6
Sorting harga berhasil
Press any key to continue . . .
```

```
DATA PERUMAHAN
--------------------------------------------------------------------------
No | Nama Developer | Nama Perumahan | Lokasi | Jumlah Unit | Harga Rumah
--------------------------------------------------------------------------
1 | efad | fass | gsfa | 2141 | 190000000
2 | faga | qte | ewhd | 1263 | 170000000
3 | hdew | ytr | rtrur | 1352 | 165000000
--------------------------------------------------------------------------
Total Data : 3
Press any key to continue . . .
```

Sorting Harga Perumahan

```
SISTEM MANAJEMEN
1.  Tambah Data
2.  Tampilkan Data
3.  Ubah Data
4.  Hapus Data
5.  Sorting Nama (Ascending)
6.  Sorting Harga (Descending)
7.  Sorting Jumlah Unit (Ascending)
8.  Cari Nama Developer (Sequential Search)
9.  Cari Jumlah Unit (Binary Search)
10. Keluar
Pilih Menu : 7
Sorting unit berhasil
Press any key to continue . . .
```

```
DATA PERUMAHAN
--------------------------------------------------------------------------
No | Nama Developer | Nama Perumahan | Lokasi | Jumlah Unit | Harga Rumah
--------------------------------------------------------------------------
1 | faga | qte | ewhd | 1263 | 170000000
2 | hdew | ytr | rtrur | 1352 | 165000000
3 | efad | fass | gsfa | 2141 | 190000000
--------------------------------------------------------------------------
Total Data : 3
Press any key to continue . . .
```

Sorting Jumlah Unit

```
CARI DATA BERDASARKAN NAMA DEVELOPER
Masukkan nama developer yang dicari: efad
Data ditemukan pada nomor 3
Nama Developer  : efad
Nama Perumahan  : fass
Lokasi          : gsfa
Jumlah Unit     : 2141
Harga Rumah     : 190000000
Press any key to continue . . .
```

Cari Nama Developer

```
CARI DATA BERDASARKAN JUMLAH UNIT RUMAH
Masukkan jumlah unit yang dicari: 1263
Data ditemukan!
Nama Developer  : faga
Nama Perumahan  : qte
Lokasi          : ewhd
Jumlah Unit     : 1263
Harga Rumah     : 170000000
Press any key to continue . . .
```

Cari Jumlah Unit Rumah

5. Langkah-langkah GIT

```
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git init
Initialized empty Git repository in C:/Users/Lucius/Documents/Praktikum-Semester-2/APL/.git/
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git add .
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git commit -m "First Upt ✅"
[master (root-commit) 2c92f6d] First Upt ✅
 2 files changed, 82 insertions(+)
 create mode 100644 Praktikum-APL/post-test/2509106019-HeizaRizki-PT-1.cpp
 create mode 100644 Praktikum-APL/post-test/2509106019-HeizaRizki-PT-1.exe
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git branch -M main
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git remote add origin https://github.com/01001100-KMS/APL.git
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 48.00 KiB | 2.00 MiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/01001100-KMS/APL.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\Lucius\Documents\Praktikum-Semester-2\APL>
```

5.1 GIT Add

```
PS C:\Users\Lucius\Documents\New folder> git add .
```

Fungsinya**:** Memilih file yang sudah diubah untuk masuk ke staging **area** (daftar siap commit). Tanpa git add, perubahan tidak akan ikut tersimpan saat git commit.

Contoh:
git add index.html
→ hanya file index.html yang siap di-commit.
git add .
→ semua file yang berubah akan masuk staging area.

5.2 GIT Commit

```
PS C:\Users\Lucius\Documents\New folder> git commit -m "ini commit"
[main 94354ce] ini commit
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Fungsinya: Menyimpan perubahan yang sudah dipilih (staging area*) ke dalam riwayat repository Commit ini ibarat checkpoint atau simpan versi dari proyek. Biasanya commit disertai pesan (-m) agar jelas maksud perubahannya. Contoh:*

commit - git m "Menambahkan fitur login"

5.3 GIT Push

Fungsinya: Mengirim perubahan (commit) yang ada di repository lokal ke repository remote (misalnya GitHub, GitLab, Bitbucket). Supaya bisa push, biasanya harus sudah git remote add origin <url> terlebih dahulu.

Contoh:
 git push origin main

→ Mengirim commit lokal ke branch main di repository remote bernama origin.